COMPTE RENDU TP2 IOT

ETUDIANT: HASSEN CHLADY ET MAZEN ELHENI

CLASSE: LSI-A2-02-02

PHASE 1 : Configuration et architecture

Question 1 : Création de la structure projet

Objectif : Mettre en place l'arborescence du projet et comprendre son organisation.

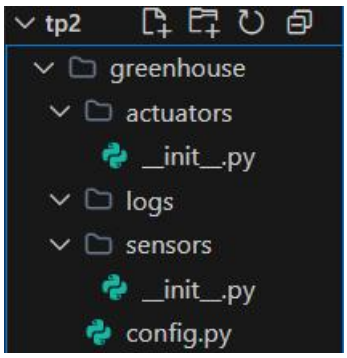
Structure attendue :

```
greenhouse/
├── config.py
├── sensors/
│   └── __init__.py
├── actuators/
│   └── __init__.py
└── logs/
```

Consigne :

a) Créez l'arborescence complète du projet greenhouse/

b) Créez les fichiers __init__.py dans sensors/ et actuators/



c) Pourquoi séparer les capteurs et actionneurs dans des dossiers distincts ?

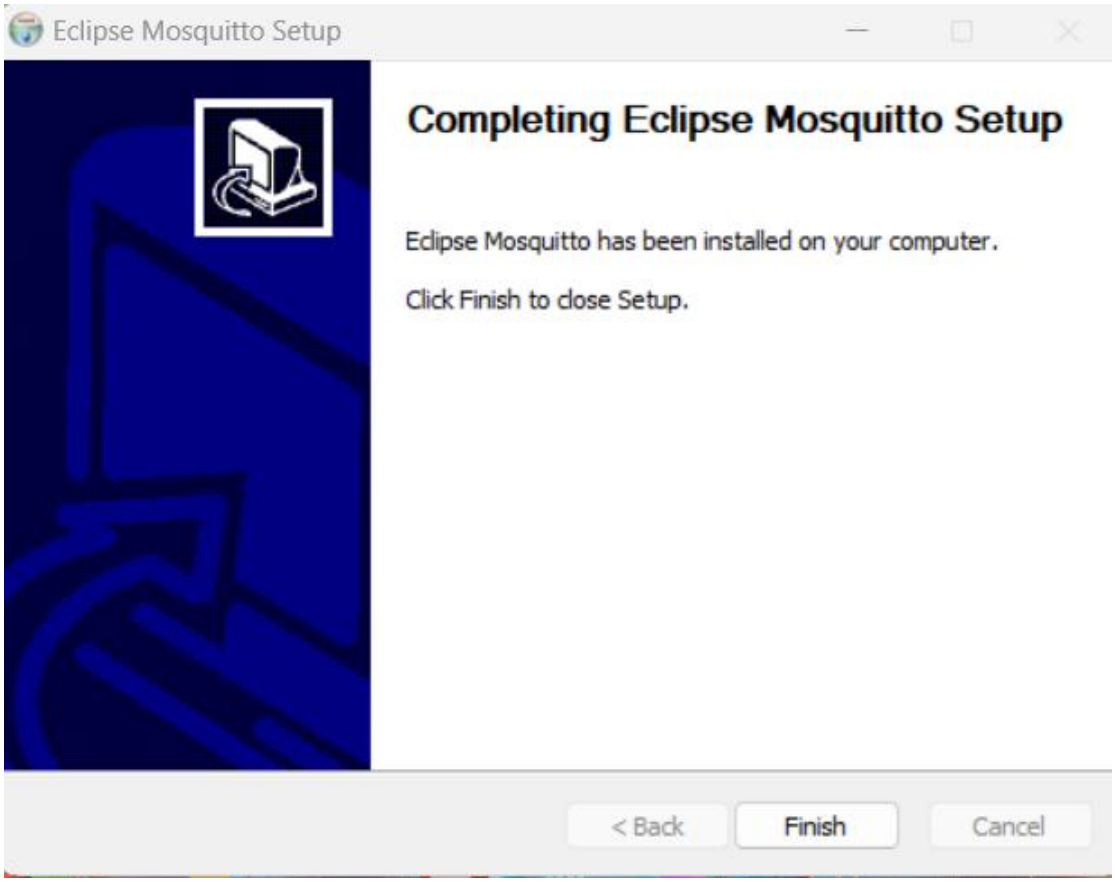
- Séparation des responsabilités/Maintenance/Réutilisabilité et tests...

Question 2 : Fichier config.py centralisé

Objectif : Centraliser toutes les configurations du système pour faciliter la maintenance.

Consigne :

a) Installez Mosquitto sur Windows (https://mosquitto.org/download/)



b) Démarrez Mosquitto dans PowerShell en exécutant : `& "C:\Program Files\Mosquitto\mosquitto.exe" -v`

```
PS C:\> cd "C:\Program Files\Mosquitto"
PS C:\Program Files\Mosquitto> .\mosquitto.exe -v
1772449628: mosquitto version 2.1.2 starting
1772449628: Using default config.
1772449628: Bridge support available.
1772449628: Persistence support available.
1772449628: TLS support available.
1772449628: TLS-PSK support available.
1772449628: Websockets support available.
1772449628: Starting in local only mode. Connections will only be possible from clients running on this machine.
1772449628: Create a configuration file which defines a listener to allow remote access.
1772449628: For more details see https://mosquitto.org/documentation/authentication-methods/
1772449628: Opening ipv4 listen socket on port 1883.
1772449628: Error: Une seule utilisation de chaque adresse de socket (protocole/adresse rÚseau/port) est habituellement
autorisÚe.

1772449628: Opening ipv6 listen socket on port 1883.
1772449628: Error: Une seule utilisation de chaque adresse de socket (protocole/adresse rÚseau/port) est habituellement
autorisÚe.

1772449628: mosquitto version 2.1.2 terminating
PS C:\Program Files\Mosquitto>
```

c) Ajoutez le code suivant au fichier greenhouse/config.py :

```
# greenhouse/config.py
```

```
"""
Configuration centrale du système IoT Serre Intelligente
Toutes les constantes du projet doivent être définies ici.
"""

# ============================================================
# MQTT BROKER
# ============================================================

BROKER_HOST = "localhost" # au lieu de "test.mosquitto.org"
BROKER_PORT = 1883
BROKER_KEEPALIVE = 60

# QoS
QOS_DEFAULT = 0
QOS_CRITICAL = 1   # commandes importantes

# Retained messages
RETAIN_STATE = True


# ============================================================
# TOPICS MQTT
# ============================================================

TOPICS = {

    # ---- Capteurs ----
    "sensors": {
        "temp_soil": "greenhouse/sensors/temp_soil",
        "humidity": "greenhouse/sensors/humidity",
        "light": "greenhouse/sensors/light",
        "water": "greenhouse/sensors/water_level"
    },

    # ---- Actionneurs ----
    "actuators": {
        "irrigation": {
            "cmd": "greenhouse/actuators/irrigation/cmd",
            "state": "greenhouse/actuators/irrigation/state"
        },
        "lighting": {
            "cmd": "greenhouse/actuators/lighting/cmd",
```

```
            "state": "greenhouse/actuators/lighting/state"
        }
    },

    # ---- Alertes ----
    "alerts": "greenhouse/alerts"
}


# ============================================================
# SEUILS MÉTIER (Business Rules)
# ============================================================

THRESHOLDS = {
    "humidity_low": 40,      # %
    "water_min": 20,         # %
    "light_low": 300,        # lux
    "temp_critical": 32      # °C
}


# ============================================================
# INTERVALLES DE PUBLICATION (secondes)
# ============================================================

SENSOR_INTERVALS = {
    "temp_soil": 5,
    "humidity": 7,
    "light": 6,
    "water": 10
}


# ============================================================
# PLAGES DE SIMULATION
# ============================================================

SENSOR_RANGES = {
    "temp_soil": (15.0, 35.0),
    "humidity": (20.0, 90.0),
    "light": (0, 1000),
    "water": (0.0, 100.0)
}
# ============================================================
```

```
# CONFIGURATION TEMPORELLE (Règle éclairage)
# ==========================================================
DAY_START_HOUR = 8
DAY_END_HOUR = 18
```

```
PS D:\iot\tp2> python .\greenhouse\config.py
PS D:\iot\tp2>
```

d) Quels sont les avantages d'avoir un fichier config.py centralisé plutôt que des constantes dispersées dans chaque fichier ?

- Point unique de configuration/Facilité de modification/Cohérence et maintenance...

PHASE 2 : Capteurs modulaires

Question 3 : Classe mère BaseSensor

Objectif : Créer une classe mère réutilisable pour éviter la duplication de code.

Consigne :

a) Créez le fichier greenhouse/sensors/base_sensor.py avec le code suivant :

```
# greenhouse/sensors/base_sensor.py


import json
import time
from datetime import datetime
import paho.mqtt.client as mqtt


from greenhouse.config import (
    BROKER_HOST,
    BROKER_PORT,
    BROKER_KEEPALIVE,
    QOS_DEFAULT
)



class BaseSensor:


    def __init__(self, sensor_id, topic, interval):
```

```
        self.sensor_id = sensor_id
        self.topic = topic
        self.interval = interval

        # Client MQTT (compatible Paho v2)
        self.client = mqtt.Client(client_id=f"sensor_{sensor_id}",
protocol=mqtt.MQTTv311)

        self.client.on_connect = self._on_connect
        self.client.on_disconnect = self._on_disconnect

        # Reconnexion automatique
        self.client.reconnect_delay_set(min_delay=1, max_delay=10)

        self.running = False
        self.message_count = 0

    # ============================================================
    # Callbacks MQTT
    # ============================================================

    def _on_connect(self, client, userdata, flags, rc):
        if rc == 0:
            print(f"[{self.sensor_id}] ☑ Connecté au broker")
        else:
            print(f"[{self.sensor_id}] ✕ Échec connexion (code {rc})")

    def _on_disconnect(self, client, userdata, rc):
        if rc != 0:
            print(f"[{self.sensor_id}] ⚠ Déconnexion inattendue - tentative
de reconnexion...")

    # ============================================================
    # Méthodes à implémenter dans les sous-classes
    # ============================================================
```

```
    def read_value(self):
        raise NotImplementedError("read_value() doit être implémentée")

    def get_unit(self):
        return ""

    # ============================================================
    # Publication MQTT
    # ============================================================

    def publish_data(self):
        try:
            value = self.read_value()

            payload = {
                "sensor_id": self.sensor_id,
                "value": value,
                "unit": self.get_unit(),
                "timestamp": datetime.now().isoformat()
            }

            result = self.client.publish(
                self.topic,
                json.dumps(payload),
                qos=QOS_DEFAULT
            )

            if result.rc == mqtt.MQTT_ERR_SUCCESS:
                self.message_count += 1
                print(f"[{self.sensor_id}] #{self.message_count} -> {value}
{self.get_unit()}")
            else:
                print(f"[{self.sensor_id}] ❌ Erreur publication")

        except Exception as e:
            print(f"[{self.sensor_id}] ❌ Exception: {e}")
```

```
    # ============================================================
    # Lifecycle
    # ============================================================

    def start(self):
        try:
            print(f"[{self.sensor_id}] Connexion au broker
{BROKER_HOST}:{BROKER_PORT}...")

            self.client.connect(BROKER_HOST, BROKER_PORT, BROKER_KEEPALIVE)
            self.client.loop_start()

            print(f"[{self.sensor_id}] 🚀 Capteur démarré")
            print(f"[{self.sensor_id}] Topic: {self.topic}")
            print(f"[{self.sensor_id}] Intervalle: {self.interval}s\n")

            self.running = True

            while self.running:
                self.publish_data()
                time.sleep(self.interval)

        except KeyboardInterrupt:
            print(f"\n[{self.sensor_id}] ⏹ Arrêt demandé par l'utilisateur")

        finally:
            self.stop()

    def stop(self):
        if self.running:
            self.running = False

        self.client.loop_stop()
        self.client.disconnect()
```

```
        print(f"[{self.sensor_id}] 🛑 Arrêté ({self.message_count} messages envoyés)")
```

```
PS D:\iot\tp2> python -m greenhouse.sensors.base_sensor
PS D:\iot\tp2>
```

b) Pourquoi utiliser une classe mère plutôt que copier-coller le code pour chaque capteur ?

- Elle rassemble la logique commune (MQTT, publication, boucle) et évite de dupliquer ce code pour chaque capteur.

c) Que se passe-t-il si on appelle read_value() sur BaseSensor directement ?

- La méthode de base lève immédiatement NotImplementedError.

- C'est un contrat : on ne doit appeler cette classe qu'en la sous-classant.

d) Pourquoi utiliser QoS 0 pour les données de capteurs ?

- Envoi « au plus une fois » sans accusé : simple, léger et tolérant aux pertes.

- Les mesures sont rapidement obsolètes, la retransmission n'a pas d'intérêt.

Question 4 : Capteur température sol

Objectif : Implémenter un capteur concret en héritant de BaseSensor.

Consigne :

a) Complétez greenhouse/sensors/temp_soil.py :

```
# greenhouse/sensors/temp_soil.py
import random
import sys
sys.path.append('..')
from sensors.base_sensor import BaseSensor
from config import TOPICS, SENSOR_INTERVALS, SENSOR_RANGES

class TemperatureSoilSensor(BaseSensor):
    def __init__(self):
        super().__init__(
            sensor_id='TEMP_SOIL_01',
            topic=TOPICS['sensors']['temp_soil'],
            interval=SENSOR_INTERVALS['temp_soil']
        )
        self.min_temp, self.max_temp = SENSOR_RANGES['temp_soil']
```

```
    def read_value(self):
        return round(random.uniform(FIXME), 1)

    def get_unit(self):
        return '°C'

if __name__ == '__main__':
    sensor = TemperatureSoilSensor()
    sensor.start()
```

b) Exécutez le code en utilisant la commande `python -m sensors.temp_soil`

Résultat attendu :

```
[TEMP_SOIL_01] ✅ Connecté au broker
[TEMP_SOIL_01]   Démarré (intervalle: 5s)
[TEMP_SOIL_01]   23.7 °C
[TEMP_SOIL_01]   28.2 °C
...
```

```
PS D:\iot\tp2\greenhouse> python -m sensors.temp_soil
[TEMP_SOIL_01] Connexion au broker localhost:1883...
[TEMP_SOIL_01] 🚀 Capteur démarré
[TEMP_SOIL_01] ✅ Connecté au broker
[TEMP_SOIL_01] Topic: greenhouse/sensors/temp_soil
[TEMP_SOIL_01] Intervalle: 5s

[TEMP_SOIL_01] #1 -> 16.3 °C
```

c) Vérifiez que le message est bien publié sur MQTT ? (sous PowerShell, exécutez la commande `& "C:\Program Files\Mosquitto\mosquitto_sub.exe" -h localhost -t greenhouse/sensors/temp_soil -v`)

```
PS D:\iot\tp2\greenhouse> & "C:\Program Files\Mosquitto\mosquitto_sub.exe" -h localhost -t greenhouse/sensors/temp_soil -v
greenhouse/sensors/temp_soil {"sensor_id": "TEMP_SOIL_01", "value": 23.5, "unit": "\u00b0C", "timestamp": "2026-03-02T12:42:56.179268"}
greenhouse/sensors/temp_soil {"sensor_id": "TEMP_SOIL_01", "value": 32.0, "unit": "\u00b0C", "timestamp": "2026-03-02T12:43:01.189347"}
greenhouse/sensors/temp_soil {"sensor_id": "TEMP_SOIL_01", "value": 31.6, "unit": "\u00b0C", "timestamp": "2026-03-02T12:43:06.199473"}
```

Question 5 : Implémentation des 3 autres capteurs

Objectif : Appliquer le même pattern pour créer les capteurs manquants.

Consigne :

En vous inspirant de temp_soil.py, créez les 3 fichiers suivants :

a) greenhouse/sensors/humidity.py

- Classe : HumiditySensor

- Plage : 20-90%
- Unité : '%'

```python
# greenhouse/sensors/humidity.py
import random
import sys
sys.path.append('..')
from sensors.base_sensor import BaseSensor
from config import TOPICS, SENSOR_INTERVALS, SENSOR_RANGES

class HumiditySensor(BaseSensor):
    def __init__(self):
        super().__init__(
            sensor_id='HUMIDITY_01',
            topic=TOPICS['sensors']['humidity'],
            interval=SENSOR_INTERVALS['humidity']
        )
        self.min_humidity, self.max_humidity = SENSOR_RANGES['humidity']

    def read_value(self):
        return round((random.uniform(self.min_humidity, self.max_humidity)), 1)

    def get_unit(self):
        return '%'

if __name__ == '__main__':
    sensor = HumiditySensor()
    sensor.start()
```

b) greenhouse/sensors/light.py

- Classe : LightSensor
- Plage : 0-1000 lux (entier)
- Unité : 'lux'

config.py | base_sensor.py U | humidity.py U | light.py U

greenhouse > sensors > light.py > ...

```python
# greenhouse/sensors/light.py
import random
import sys
sys.path.append('..')
from sensors.base_sensor import BaseSensor
from config import TOPICS, SENSOR_INTERVALS, SENSOR_RANGES

class LightSensor(BaseSensor):
    def __init__(self):
        super().__init__(
            sensor_id='LIGHT_01',
            topic=TOPICS['sensors']['light'],
            interval=SENSOR_INTERVALS['light']
        )
        self.min_light, self.max_light = SENSOR_RANGES['light']

    def read_value(self):
        return int(random.uniform(self.min_light, self.max_light))

    def get_unit(self):
        return 'lux'

if __name__ == '__main__':
    sensor = LightSensor()
    sensor.start()
```

c) greenhouse/sensors/water.py

- Classe : WaterLevelSensor
- Plage : 0-100%
- Unité : '%'

```python
# greenhouse/sensors/water.py
import random
import sys
sys.path.append('..')
from sensors.base_sensor import BaseSensor
from config import TOPICS, SENSOR_INTERVALS, SENSOR_RANGES

class WaterLevelSensor(BaseSensor):
    def __init__(self):
        super().__init__(
            sensor_id='WATER_LEVEL_01',
            topic=TOPICS['sensors']['water'],
            interval=SENSOR_INTERVALS['water']
        )
        self.min_level, self.max_level = SENSOR_RANGES['water']

    def read_value(self):
        return round((random.uniform(self.min_level, self.max_level)), 1)

    def get_unit(self):
        return '%'

if __name__ == '__main__':
    sensor = WaterLevelSensor()
    sensor.start()
```

ASTUCE

Utilisez random.uniform() pour les nombres décimaux et random.randint() pour les entiers. N'oubliez pas d'importer random en haut du fichier !

d) Test :

Lancez les 4 capteurs simultanément dans 4 terminaux différents.

PHASE 2 TERMINÉE

Vous avez maintenant 4 capteurs modulaires qui publient des données JSON sur MQTT !

```
PS D:\iot\tp2\greenhouse> & "C:\Program Files\Mosquitto\mosquitto_sub.exe" -h localhost -t greenhouse/sensors/+ -v
greenhouse/sensors/humidity {"sensor_id": "HUMIDITY_01", "value": 65.7, "unit": "%", "timestamp": "2026-03-02T12:52:11.567384"}
greenhouse/sensors/light {"sensor_id": "LIGHT_01", "value": 922, "unit": "lux", "timestamp": "2026-03-02T12:52:11.900386"}
greenhouse/sensors/temp_soil {"sensor_id": "TEMP_SOIL_01", "value": 17.6, "unit": "\u00b0C", "timestamp": "2026-03-02T12:52:12.377038"}
greenhouse/sensors/water_level {"sensor_id": "WATER_LEVEL_01", "value": 62.7, "unit": "%", "timestamp": "2026-03-02T12:52:13.693446"}
```

PHASE 3 : Actionneurs intelligents

Question 6 : Classe mère BaseActuator

Objectif : Créer une classe mère pour les actionneurs avec gestion d'état et QoS 1 + Retained.

Consigne :

a) Créez le fichier greenhouse/actuators/base_actuator.py avec le code suivant :

```
# greenhouse/actuators/base_actuator.py

import paho.mqtt.client as mqtt
import json
from datetime import datetime
import sys
sys.path.append('..')
from config import BROKER_HOST, BROKER_PORT, BROKER_KEEPALIVE

class BaseActuator:
    def __init__(self, actuator_id, topic_cmd, topic_state):
        self.actuator_id = actuator_id
        self.topic_cmd = topic_cmd
        self.topic_state = topic_state
        self.state = 'OFF'

        self.client = mqtt.Client(f'actuator_{actuator_id}')
        self.client.on_connect = self._on_connect
        self.client.on_message = self._on_message
        self.client.on_disconnect = self._on_disconnect

    def _on_connect(self, client, userdata, flags, rc):
        if rc == 0:
            print(f'[{self.actuator_id}] Connecté au broker')
            client.subscribe(self.topic_cmd, qos=1)
            print(f'[{self.actuator_id}] Abonné à {self.topic_cmd}')
            self.publish_state()
        else:
            print(f'[{self.actuator_id}] Échec connexion')
```

```
    def _on_message(self, client, userdata, msg):
        try:
            cmd = json.loads(msg.payload.decode())
            action = cmd.get('action', '').upper()

            if action in ['ON', 'OFF']:
                self.state = action
                print(f'[{self.actuator_id}] Commande: {action}')
                self.execute_action(action)
                self.publish_state()
        except Exception as e:
            print(f'[{self.actuator_id}] Erreur: {e}')

    def execute_action(self, action):
        print(f'[{self.actuator_id}] Action: {action}')

    def publish_state(self):
        payload = {
            'actuator_id': self.actuator_id,
            'state': self.state,
            'timestamp': datetime.now().isoformat()
        }
        # QoS 1 + Retained pour commandes critiques
        self.client.publish(self.topic_state, json.dumps(payload), qos=1,
retain=True)
        print(f'[{self.actuator_id}] État: {self.state}')

    def start(self):
        try:
            self.client.connect(BROKER_HOST, BROKER_PORT, BROKER_KEEPALIVE)
            print(f'[{self.actuator_id}] Démarré')
            self.client.loop_forever()
        except KeyboardInterrupt:
            print('Arrêt')
        finally:
            self.client.disconnect()
```

```
PS D:\iot\tp2\greenhouse> python -m actuators.base_actuator
PS D:\iot\tp2\greenhouse>
```

b) Pourquoi utiliser QoS 1 pour les commandes des actionneurs ?

- QoS 1 garantit que la commande arrive au moins une fois avec accusé de réception.

- C'est critique : commander l'irrigation doit être sûr, contrairement aux simples mesures de capteurs.

c) Quel est l'intérêt du message Retained pour l'état d'un actionneur ?

- Un message retained persiste sur le broker et est automatiquement envoyé aux nouveaux abonnés.

- Ainsi, un client qui se connecte tard connaît immédiatement l'état actuel (irrigateur on/off) sans attendre la prochaine mise à jour.

d) Pourquoi avoir 2 topics par actionneur (cmd et state) ?

- Séparation des flux : cmd reçoit les ordres à exécuter, state publie le résultat réel.

- Cela évite les boucles, permet de vérifier que la commande a réussi, et laisse tracer toute divergence (ordre ignoré, défaillance)

BONNES PRATIQUES

QoS 1 garantit qu'une commande critique (ON/OFF) sera livrée au moins une fois. Retained permet aux nouveaux subscribers de connaître immédiatement l'état actuel.

Question 7 : Actionneur irrigation

Objectif : Implémenter l'actionneur d'irrigation avec simulation visuelle.

a) Créez le fichier greenhouse/actuators/irrigation.py avec le code suivant :

```
# greenhouse/actuators/irrigation.py

"""
Actionneur Irrigation - Serre Intelligente

Objectif :
- Recevoir commandes MQTT (ON / OFF)
- Simuler l'irrigation
- Publier son état (retained)
- Utiliser QoS critique pour les commandes importantes
"""
```

```python
# ============================================================
# PERMETTRE IMPORT DU DOSSIER PARENT (greenhouse/)
# ============================================================

import sys
sys.path.append('..')  # ajoute le dossier parent au PYTHONPATH

# ============================================================
# IMPORTS
# ============================================================

import time
import paho.mqtt.client as mqtt

from config import (
    BROKER_HOST,
    BROKER_PORT,
    BROKER_KEEPALIVE,
    QOS_CRITICAL,
    RETAIN_STATE,
    TOPICS
)

# ============================================================
# ÉTAT INTERNE
# ============================================================

irrigation_state = "OFF"  # État initial


# ============================================================
# CALLBACK CONNEXION
# ============================================================

def on_connect(client, userdata, flags, rc):
    """
    Appelé lors de la connexion au broker MQTT.
    """
    if rc == 0:
        print("☑ Irrigation connectée au broker MQTT")

        # Abonnement au topic de commande
        client.subscribe(
            TOPICS["actuators"]["irrigation"]["cmd"],
```

```
            qos=QOS_CRITICAL
        )

        print("📡 Abonné au topic commande irrigation")

        # Publier l'état initial (important si retained activé)
        publish_state(client)

    else:
        print("❌ Échec connexion MQTT")


# ============================================================
# CALLBACK RÉCEPTION MESSAGE
# ============================================================

def on_message(client, userdata, msg):
    """
    Appelé lorsqu'une commande est reçue.
    """
    global irrigation_state

    payload = msg.payload.decode().strip().upper()
    print(f"\n📥 Commande reçue : {payload}")

    # Vérifier validité commande
    if payload not in ["ON", "OFF"]:
        print("⚠️ Commande invalide (utiliser ON ou OFF)")
        return

    # Changement d'état uniquement si nécessaire
    if payload != irrigation_state:
        irrigation_state = payload

        simulate_irrigation(irrigation_state)
        publish_state(client)
    else:
        print("ℹ️ État déjà actif, aucune action")


# ============================================================
# SIMULATION VISUELLE
# ============================================================
```

```python
def simulate_irrigation(state):
    """
    Simulation simple de l'irrigation.
    """
    if state == "ON":
        print("\n💧 Irrigation ACTIVÉE")
        print("🌱 Arrosage en cours...")
        time.sleep(1)
        print("💦 Eau distribuée aux plantes\n")
    else:
        print("\n🔘 Irrigation DÉSACTIVÉE")
        print("🌱 Arrosage arrêté\n")


# ============================================================
# PUBLICATION ÉTAT (RETAINED)
# ============================================================

def publish_state(client):
    """
    Publie l'état actuel.
    Retained=True → les nouveaux abonnés reçoivent l'état immédiatement.
    """
    client.publish(
        TOPICS["actuators"]["irrigation"]["state"],
        irrigation_state,
        qos=QOS_CRITICAL,
        retain=RETAIN_STATE
    )

    print(f"📤 État publié : {irrigation_state}")


# ============================================================
# PROGRAMME PRINCIPAL
# ============================================================

def main():
    """
    Initialise le client MQTT et démarre l'écoute.
    """

    client = mqtt.Client(client_id="irrigation_actuator")
```

```
    client.on_connect = on_connect
    client.on_message = on_message

    # Connexion au broker
    client.connect(
        BROKER_HOST,
        BROKER_PORT,
        BROKER_KEEPALIVE
    )

    print("🚀 Actionneur irrigation en attente de commandes...\n")

    # Boucle infinie MQTT
    client.loop_forever()


# ============================================================
# POINT D'ENTRÉE
# ============================================================

if __name__ == "__main__":
    main()
```

b) Exécutez la commande : `python -m actuators.irrigation`

```
PS D:\iot\tp2\greenhouse> python -m actuators.irrigation
D:\iot\tp2\greenhouse\actuators\irrigation.py:143: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="irrigation_actuator")
🚀 Actionneur irrigation en attente de commandes...

✅ Irrigation connectée au broker MQTT
📡 Abonné au topic commande irrigation
📤 État publié : OFF
```

c) Dans autre terminal, exécutez la commande :

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -t
greenhouse/actuators/irrigation/cmd -m "ON"
```

```
PS D:\iot\tp2\greenhouse> & "C:\Program Files\mosquitto\mosquitto_pub.exe" -t greenhouse/actuators/irrigation/cmd -m "ON"
PS D:\iot\tp2\greenhouse>
```

```
📩 Commande reçue : ON

💧 Irrigation ACTIVÉE
🌱 Arrosage en cours...
🌊 Eau distribuée aux plantes

📤 État publié : ON
```

Question 8 : Actionneur éclairage

a) Créez greenhouse/actuators/lighting.py en vous inspirant d'irrigation.py

- Classe : LightingActuator
- ID : 'LIGHTING_01'
- Action ON : Afficher 'ÉCLAIRAGE ALLUMÉ'
- Action OFF : Afficher 'Éclairage éteint'

b) Exécutez la commande : `python -m actuators.lighting`

```
PS D:\iot\tp2\greenhouse> python -m actuators.irrigation
D:\iot\tp2\greenhouse\actuators\irrigation.py:143: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="irrigation_actuator")
🚀 Actionneur irrigation en attente de commandes...

✅ Irrigation connectée au broker MQTT
📡 Abonné au topic commande irrigation
📤 État publié : OFF
```

c) Dans autre terminal, exécutez la commande :

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -t
greenhouse/actuators/lighting/cmd -m "ON"
```

```
PS D:\iot\tp2\greenhouse> & "C:\Program Files\mosquitto\mosquitto_pub.exe" -t greenhouse/actuators/lighting/cmd -m "ON"
PS D:\iot\tp2\greenhouse>
```

```
📩 Commande reçue : ON

💧 Irrigation ACTIVÉE
⏳ Arrosage en cours...
🌊 Eau distribuée aux plantes

📤 État publié : ON
```